



asishallab-group / Tensor-Omics ▾



Code **Issues** 38 Pull requests 11 Agents Discussions Actions Projects

Security and quality Insights Settings

New issue



Implementation of Generalized Operators for Ensemble Growth and Clustering #136

Open

Assignees



Specter1999 opened on May 15

Collaborator



Generalized functions for "operators"

1. Coarse graining or better `increase_ensemble` :

- **Arguments:** set of vectors (ensemble) probably as a matrix or a logical selecting column vectors (indices) of "ambient_space", ambient space (the superset of all vectors), further arguments
 - There can be different functions that we implement here like density based, energy based, angular based...
- **Return value (output):** A grown ensemble or the same if growth was rejected

2. Feature

- **Arguments:** An ensemble and optional further arguments
- **Returns:** A real vector representing the ensemble without scale
 - Can be: Density, Angular Variance, First Singular Vector, Manifold, Free Energy, Rank,

3. Normalization

- As of now, I'm unsure whether we need this or whether this will be an integrated part of the Feature function family
- **Arguments:** An ensemble (size), the feature,
- **Returns:** The feature normalized to the size, i.e. number of member vectors of the Ensemble

Clustering algorithm implementation:

1. Starts with an initial **Ensemble**
2. Gets its **feature**
3. **Grows** the ensemble
4. Gets the **feature** for the grown ensemble
5. Checks the **difference** between Feature F_{i+1} and F_i , potentially using the first derivative
6. **Outputs** the iterations results:
 - **Ensemble and their features**
 - **Flow of Feature differences**

[Create sub-issue](#)

  **Specter1999** assigned [asishallab](#) and [VivianBass](#) on May 15

  **VivianBass** assigned [Specter1999](#) and unassigned [asishallab](#) and [VivianBass](#) on May 21



asishallab on May 28

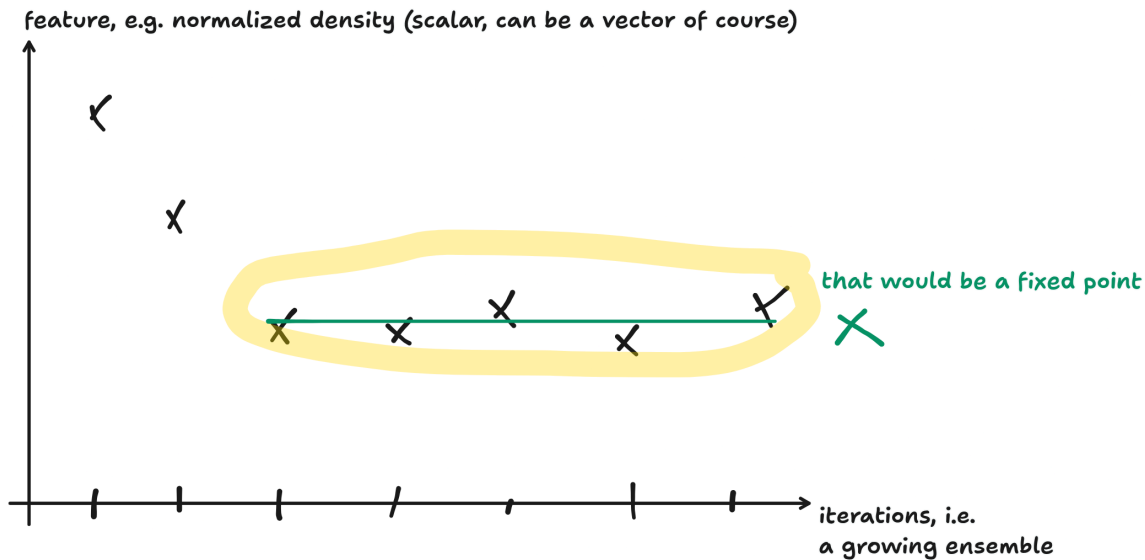
Member



Meeting notes from May 28th 2026

- We will need a function to get starting ensembles. In our current approach based on density labels this would be the top t (parameter) most dense vectors.
 - Bear in mind we try to achieve two things at this moment:
 - A general and "interface like" definition of how to do physics inspired clustering (renormalization and coarse graining)
 - A first (test) implementation based on density
- For what a fixed point is see the below sketch, please
- We will need a function to define a "neighborhood", so a subset of the ambient space that contains *candidate* vectors to be added to an existing ensemble. This potentially can be part of the "grow" function

Asis' understanding of physics inspired fixed points:



Specter1999 on Jun 13

Collaborator

Author

...

1. Coarse graining or better increase_ensemble:

For our ensemble expansion operator $E_{n+1} = G(E_n)$, I am currently using a dual-gated intersection filter to accept new candidate vectors based on spatial proximity to the moving centroid (distance $\leq r$) and a minimum local density (density $\geq \text{min_density_threshold}$).

Do you think this baseline logic is sufficient for high-dimensional omics profiles, or should we use something different ?

2. Feature

I implemented a Coordinate-Wise Dispersion Operator using scale-invariant Statistical Variance along each independent data dimension. This yields a feature vector with a fixed length equal to $n_{\text{dimensions}}$, allowing us to cleanly track the downstream derivative flow ($\Delta F = |f_{n+1} - f_n|$) across iterations to detect phase transitions.

Could you please let me know if this variance-based dispersion profile is the correct approach for our framework, or is there another specific method you would prefer to see

3. Normalization

For our descriptor flow analysis, I am tracking step-by-step changes by calculating the Euclidean distance of the discrete first derivative vector ($|f_{i+1} - f_i|$) across iterations.

Could you please confirm if this L_2 norm approach is correct for identifying our structural cluster plateaus, or is there an alternative difference metric you would prefer we use?



asishallab on Jun 16

Last edited by asishallab

Member



Thank you [@Specter1999](#) for the feedback.

Answering your questions, below:

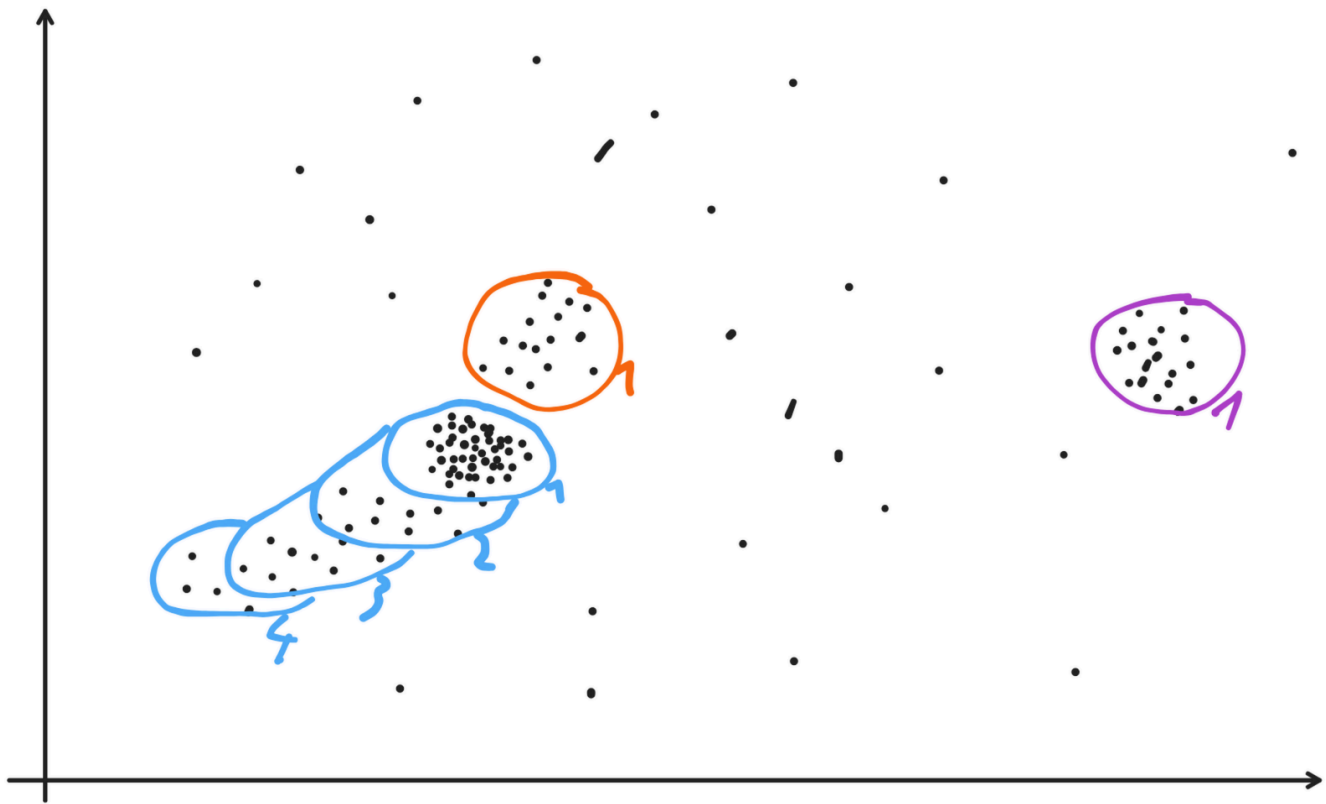
1. Coarse graining or better increase_ensemble

I agree *almost* fully. I'd not use a radial distance from the ensemble's centroid (mean ensemble vector) but use our K-D-Tree to find vectors not member of the ensemble but in the radial distance of ensemble members, i.e. grow the ensemble at its surface. This will avoid the problem of by preference generating spherical clusters as in k-means or Markov Clustering.

So, in the beginning, you start with ensembles consisting of single vectors, selected as the top t percentile of density labels. Then in the iterative growing consider the surface of the ensemble by using the k-d-tree to find vectors not yet in the ensemble but in the max allowed distance to ensemble vectors (d) *and* meeting the density selection criteria.

Importantly, we grow at the surface without the need to identify the surface, which in higher dimensions is not trivial. We use our k-d-tree. Think about a clever way to do so, maybe a logical mask of ambient vectors that meet the density criteria *and* not in the ensemble combined with the k-d-tree or something similar might work. Be careful not to *preselect* ambient vectors. We need to observe the change of the feature (density) while growing and aim at scale invariance. A preselection might introduce too strict a rule. Then, on the other hand it might not. Only experiments will show ;-)

2. Feature



In the above example we already note a few things:

- Density is a good first implementation feature. Later we want to add more features. But for now, density gives a scalar value (length one vector) and already is number of data points (vectors) per volume. We approximate this by either
 - calculate the mean ensemble vector, get the max distance (r_m) to any ensemble member, use this r_m to calculate a hyper-spherical volume and then calculate the density as no. of ensemble elements / volume
 - or mean(ensemble density labels)

So, in our API, as we are thinking it should be at some point, an ensemble feature function should receive as input the ensemble (logical mask of ambient vectors maybe) and calculate a feature vector.

3. Normalization

That's an excellent question. And I don't have a definitive answer. We potentially should get together with Prof. Schäfer or Prof. Duque-Anton to discuss these things. On the other hand this might need experiments. So, program it in such a way, that we can easily select which function to use to calculate the difference for features between two subsequent iterations ($\Delta(f_{i+1}, f_e)$, which can be $\|f_{i+1} - f_i\|_2$). For now I would start with the Euclidean norm.

We can provide arguments to each of our functions that actually select e.g. the function to use as Δ , the feature function, etc..

Stop Condition

Also here, we will need to experiment and allow the user to set or select from a set of different stop conditions. Stop if,

- min value is reached, e.g. median of background density (feature), or
- max iterations have been carried out, or
- a plateau has been reached, i.e. i iterations have not changed the feature more than a first derivative equivalent to tangential 10° *and* the next iteration did change it more. So, go back one iteration and that's the scale invariant ensemble.

All of this needs to be experimented on.

Simulated data for testing

[@VivianBass](#) could you share your data-sets which we used for our "smoothing" (manifold learning) experiments and maybe extend them to 3,4,5... dimensions using the same generation rules?



Specter1999 1mo ago

Collaborator

Author



KD-tree surface growth from existing ensemble members (not centroid-based), accepting only non-members within distance d that pass the density threshold, and returning unchanged ensemble when no growth is accepted? Also, should top- t density singleton seed selection remain strictly outside this operator as a separate initialization step?



asishallab 1mo ago

Member



Thank you [@Specter1999](#) and [@VivianBass](#) for this feedback. I will answer the two questions asked in [this comment](#) below:

1. question

KD-tree surface growth from existing ensemble members (not centroid-based), accepting only non-members within distance d that pass the density threshold, and returning unchanged ensemble when no growth is accepted?

- Make sure the argument d is exposed as an optional function parameter and has a reasonable default value
- When the ensemble is not grown one of the stop conditions has been fulfilled and the cluster (ensemble) should be returned as grown to the max - so to say

2. question

Also, should top-t density singleton seed selection remain strictly outside this operator as a separate initialization step?

Yes, I believe without being sure. As long as we avoid violating the DRY (Don't repeat yourself) principle.

I think our singleton statistical physics ensemble / scale invariant feature detector interface should have the following functions:

1. Initial seed detector
2. Ensemble feature(ensemble, ...) -> feature vector. Potentially this feature should already be normalized, i.e. independent of ensemble size / cardinality.
3. (optional) normalize_feature function to make a feature independent of ensemble size / cardinality / scale
4. Ensemble growth

In fact the function ensemble growth should *potentially* get access to the feature, because an ensemble is only grown if the feature remains scale invariant. We need to be careful here, though, and not introduce circular reasoning.



Specter1999 3w ago

Collaborator

Author



2. Feature

To ensure numerical stability in high-dimensional spaces, should we prioritize averaging our pre-calculated density labels over hyper-spherical volume calculations, and how should we handle the zero-radius edgecase for single-vector starting seeds? Also, do you prefer we detect cluster boundaries using a simple density drop threshold or by tracking the inflection point where the rate of density change peaks?

3. Normalization

To make the feature difference (Δ) and stopping conditions easily selectable for your upcoming experiments, should we implement them using a simple integer-based method selector in our API? Also, for the plateau detection, should the pipeline cache the iteration history so we can cleanly roll back one step once the tangential slope exceeds 10 degrees?



asishallab 3w ago

Last edited by asishallab

Member



First, in the codebase, the documentation, and the manual, let's please call the feature(s) from now on the "observable", because apparently that is what it is called in statistical physics.

Answers to [your questions](#) below.

Feature

Our first test-observable (feature) is density. Density in this context I'd define as number of vectors per volume. This is a relative measure, and thus automatically scale normalized.

should we prioritize averaging our pre-calculated density labels over hyper-spherical volume calculations

So, the question is whether to calculate the volume of the (growing) ensemble and then its density as number-of-vectors divided by volume. Or alternatively take e.g. the mean of the ensemble's density labels. This is a good question. In fact calculating the volume, and surface for that matter, of a given ensemble is far from trivial. Actually in multivariate high dimensional data science this is hard problem. So, we need to approximate the volume of our ensemble. We can do that by several ways.

Implementation. Change or replace the current observable (feature) function to:

```
pure function feature(ensemble_mask, density_labels) -> length-2-real-array {
  1. get the density labels of the ensemble. The labels should ideally be organized as
  the ambient vectors, i.e. the column index in ambient vectors (ensemble_mask) should match
  the column indices in labels
  2. calculate the following mean density values:
    * arithmetic mean -> mean_arithmetic_density
    * harmonic mean -> mean_harmonic_density
  3. return [mean_harmonic_density, mean_arithmetic_density]
}
```

Write tests for the above.

Make sure, we *track* the trajectory of observables through the iterations, i.e. we get a 2D array where each column index is the iteration and each column vector is the above feature. Define a max number of iterations to know how many columns you need. Or let's only track the *o* last observables.

Normalization

To make the feature difference (Δ) and stopping conditions easily selectable for your upcoming experiments, should we implement them using a simple integer-based method selector in our API? Also, for the plateau detection, should the pipeline cache the iteration history so we can cleanly roll back one step once the tangential slope exceeds 10 degrees?

This question is about how to define the stop condition. So, in our inspiration the statistical physics - as far as [@asishallab](#) understands it - normalization actually is the conversion of an observable (feature) to a relative measure, i.e. a measure that is independent of the ensemble size (number of vector elements). So probably we should define another API function or rename this to `accept_ensemble` or `growth_limit_reached` or something alike.

For our first default density-based experiment, I'd correct the original definition of the above normalization, which should now be called e.g. `accept`.

Acceptance Criterion

The function name ``accept()`` is retained.

Rationale:

- Bobito performs **atomic growth**, i.e. an entire candidate set is added in one coarse-graining step.
- If the newly grown ensemble is **not accepted**, the growth limit has been reached.
- The algorithm therefore returns the **previous iteration's ensemble** $\mathcal{E}_t$ as the final module.

Observable Trajectory

Observables are stored as a two-dimensional real array:

```
real(dp) :: observables(n_observables, n_iterations)
```

where column ``i`` corresponds to growth iteration ``i`` and contains

Index	Observable
1	Arithmetic mean density ρ_{arith}
2	Harmonic mean density ρ_{harm}
3	Density heterogeneity $H_{\rho} = \rho_{\text{arith}} / \rho_{\text{harm}}$
4	Ensemble size $ \mathcal{E} $
5	Number of candidates $ C $

Note: In principle, H_{ρ} is redundant, as it can be computed from the first two observables. It is nevertheless stored for convenience and debugging.

``accept()``

logical function `accept(observables, ...)`

The function receives only

- the observable trajectory,
- optional arguments with default values.

The function returns a boolean decision.

The current growth step is **rejected** if either

- the absolute base-2 log-fold change of the density heterogeneity satisfies

$$\left| \right|$$

```

\log_2
\left(
\frac{H_{\rho,t+1}}{H_{\rho,t}}
\right)
\right|
\geq 1
$$

```

(default: two-fold change),

- or the maximum number of growth iterations has been reached.

If ``accept()`` rejects the current growth step, the algorithm returns the previous ensemble

```

$$
\mathcal{E}_t
$$

```

as the final shape-truthful cluster.

The above renders to:

Acceptance Criterion

The function name `accept()` is retained.

Rationale:

- Bobito performs **atomic growth**, i.e. an entire candidate set is added in one coarse-graining step.
- If the newly grown ensemble is **not accepted**, the growth limit has been reached.
- The algorithm therefore returns the **previous iteration's ensemble** $\mathcal{E}_t$ as the final module.

Observable Trajectory

Observables are stored as a two-dimensional real array:

```
real(dp) :: observables(n_observables, n_iterations)
```

where column i corresponds to growth iteration i and contains

Index	Observable
1	Arithmetic mean density ρ_{arith}
2	Harmonic mean density ρ_{harm}
3	Density heterogeneity $H_\rho = \rho_{\text{arith}}/\rho_{\text{harm}}$
4	Ensemble size $ \mathcal{E} $
5	Number of candidates $ C $

Note: In principle, H_ρ is redundant, as it can be computed from the first two observables. It is nevertheless stored for convenience and debugging.

`accept()`

```
logical function accept(observables, ...)
```

The function receives only

- the observable trajectory,
- optional arguments with default values.

The function returns a boolean decision.

The current growth step is **rejected** if either

- the absolute base-2 log-fold change of the density heterogeneity satisfies

$$\left| \log_2 \left(\frac{H_{\rho,t+1}}{H_{\rho,t}} \right) \right| \geq 1$$

(default: two-fold change),

- or the maximum number of growth iterations has been reached.

If `accept()` rejects the current growth step, the algorithm returns the previous ensemble

$$\mathcal{E}_t$$

as the final shape-truthful cluster.





Generalized functions for "operators"

1. Calculating density radius (`calculate_density_radius`)

- Calculate the global arithmetic mean vector.
- Compute the Euclidean distance from every vector to the mean vector.
- Select the distance at the 15th percentile (configurable) as the search radius.

2. Calculating local density (`calculate_labels_as_density`)

- Build a K-D tree.
- Perform parallel range searches using the search radius.
- Count neighboring vectors to calculate the local density of each vector.

3. Initial seed detection (`initial_seeds`)

- Sort all vectors by local density (ascending) using heapsort.
- Select the top t -percentile (configurable) most dense vectors as initial seed candidates.
- Return the permutation index for vectors sorted by density.

4. Surface expansion (`increase_ensemble`)

- Perform parallel K-D tree range searches around active ensemble members.
- Find neighboring vectors adjacent to the current ensemble.
- Add neighboring vectors that satisfy the minimum density threshold.

5. Computing ensemble observables (`compute_ensemble_observable`)

- Compute the arithmetic mean density (ρ_{arith}).
- Compute the harmonic mean density (ρ_{harm}).
- Compute the density heterogeneity (H_{ρ}).
- Return the observable vector:
 - ρ_{arith}
 - ρ_{harm}
 - H_{ρ}
 - $|E|$
 - $|C|$

6. Acceptance decision (`accept_ensemble`)

- Track the evolution of density heterogeneity (H_ρ).
- Reject the current expansion if the absolute $\log_2$ fold change of $H_\rho \geq 1.0$ (2-fold change).
- Stop if the maximum number of iterations is reached.

7. Multi-cluster extraction (`extract_clusters`)

- Traverse candidate seeds from highest to lowest density.
- Start a new cluster from each unassigned seed.
- Expand the cluster step-by-step using `increase_ensemble`.
- Evaluate the cluster after each expansion using `accept_ensemble`.
- Roll back to the previous valid state if an expansion is rejected.
- Assign the final cluster ID to all accepted members.



asishallab 2w ago

Last edited by asishallab

Member



Thank you [@Specter1999](#) for your clarifications. Please find feedback below.

Initial seed detection (`initial_seeds`)

Return the permutation index for vectors sorted by density.

Return the top t -percentile of the vectors based on their density labels, and return the vectors sorted by density label descending.

Surface expansion (`increase_ensemble`)

An open question is what to use as a search radius for each ensemble vector. I'd suggest to use the radius r from step 1 as a default. Make it an optional argument for the user, though.

Candidate vectors are considered density-compatible if their density label differs from the current ensemble median density by at most $\alpha \cdot \text{MAD}_{\text{ambient}}$, where MAD is the Median Absolute Deviation of all ambient density labels. Unlike the standard deviation, MAD is robust to outliers and skewed density distributions, making it well suited for heterogeneous biological manifolds.

$$\text{MAD}_{\text{ambient}} = \text{median}_{i=1, \dots, N} \left(\left| \rho_i - \text{median}_{j=1, \dots, N}(\rho_j) \right| \right)$$

where ρ_i is the precomputed density label of ambient vector i . This makes it explicit that the MAD is computed once over all ambient vectors.

An input parameter is α_{MAD} the factor with which to multiply the density MAD to infer the max *absolute* deviation from the current ensemble's arithmetic mean density. Test $\alpha_{\text{MAD}} \in 0.25, 0.5, 0.75, 1$

Computing ensemble observables (`compute_ensemble_observable`)

The only thing I need to add here is to track the **last** $t_{\text{observables}}$ (input parameter; default e.g. 10) vectors. That means that for each ensemble store a two dimensional Fortran array, where each column is *one* observable vector and the column index matches the iteration index. Allow the user to specify $t_{\text{observables}} = \infty$ to track *all* observables.

Acceptance decision (`accept_ensemble`)

We *must* test different acceptance thresholds. I suggest to $\log_2(\frac{H_{\rho,t+1}}{H_{\rho,t}}) \geq \alpha_{\text{accept}} \in (0.2, 0.4, \dots, 1, \dots, 1.5)$. Thus α_{accept} is an optional input parameter with default value set to e.g. 0.5.

Multi-cluster extraction (`extract_clusters`)

If this is already implemented. Run it and show us the results. It's almost guaranteed to be adjusted during our experimentation process.

My intuition is to not run ensemble growth on each ambient vector that has not been assigned to a cluster, but rather consider having "background noise" points that are not assigned to any cluster. Or rather, make this a configurable (user input) decision.

Also, there must be a final step where we merge final ensembles (clusters) based on set-intersection. Also here, at some point the user must be able to decide the behavior of our algorithm. By default, I'd suggest to merge ensemble's if and only if they at least share one ambient vector. But the min. number of points in the intersection or a min Jaccard-Similarity-Index should become future optional user inputs.

In fact what you are describing is a main recursion / iteration loop. We have an iterative approach for each ensemble, starting from the seed to the final accepted ensemble. But there's also a `main` function that decides whether to look for more ensembles after having started with the seeds and their coarse-graining has terminated. The question here is to aim at assigning all, most, or only some vectors to ensembles. This should be a user decision, I believe.

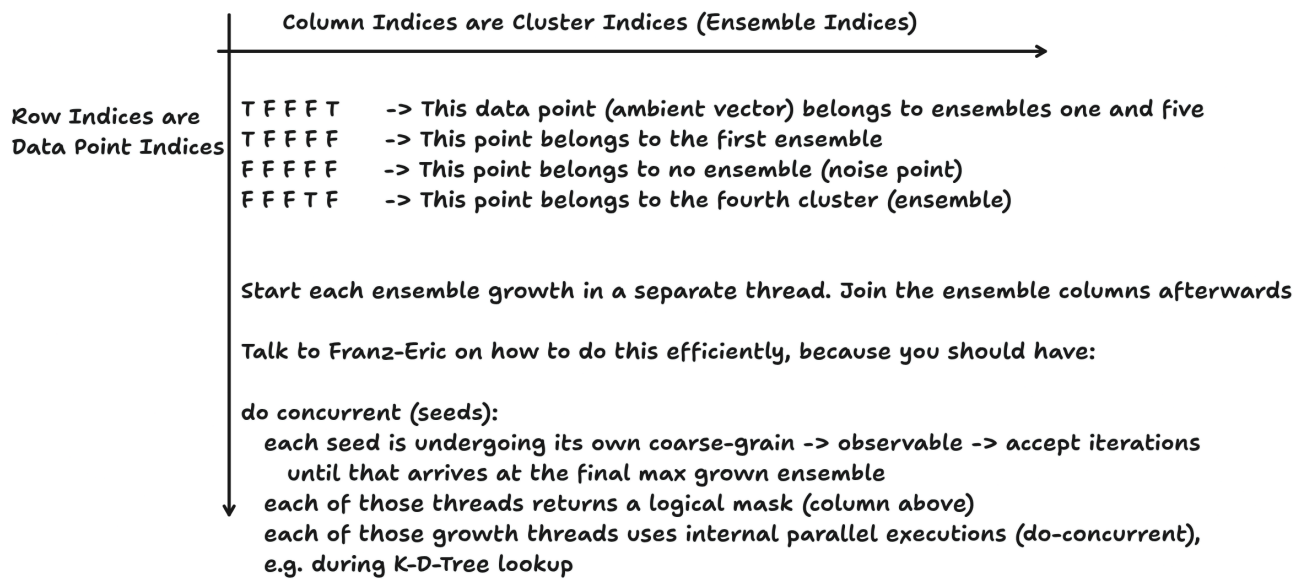
Our shape truthful clustering (STC) algorithm has one central output which is a two dimensional array of logical masks, where each logical mask is a *column* vector in that two dimensional Fortran array (matrix). The logical mask has the same length as the ensemble vectors and it indicates which vectors (row-index) belong to which ensemble / cluster (column-index). Note that in our current approach, a vector might be assigned to several clusters, accept we do merging at the end, which depends on the user input. By default I'd suggest to do merging.

Renaming of functions

In order to stay close to the statistical physics inspiration please rename some functions:

- `increase_ensemble` -> `grow_ensemble` (or `coarse_grain`)

Sketch of the final data structure and global algorithm parallelization



Note, I would use thread parallelization on the outer loop (the independent construction of ensemble logical masks, one per seed). Within each thread, i.e. within each ensemble growth (iterate until no longer accepted coarse-grain -> observable -> accept), we should use vectorization. This should be decided *automatically* by the compiler. Just by providing the right flag it should be able to use threads for outer `do concurrent` loops. Talk to [@qwerdenkerXD](#) to get input on how to run the compiler here.



asishallab 2w ago

Member



Requested renaming of Shape Truthful Clustering API Functions

- `initial_seeds` should be named `identify_ensemble_seeds`

- `increase_ensemble` should be named `grow_ensemble`
- `extract_clusters` should be `obtain_ensembles`

The coarse graining API should be:

- `grow_ensemble()`
constructs a larger candidate microscopic ensemble
- `compute_ensemble_observable()`
coarse-grains that ensemble into a small effective description
- `accept_ensemble()`
tests whether the effective description remains sufficiently stable



asishallab 1w ago

Last edited by asishallab ▾

Member



Some remarks on the observable tracking

To save memory and probably, because we do not need to track the evolution of observables through all iterations, just the last $t_{\text{observable}}$ with a default of e.g. 10 might be enough.

So, the question arises of how to implement this cleanly.

I suggest to use a two dimensional array. Allocate a two dimensional array $\mathcal{O}$ of $t_{\text{observable}}$ columns and as many rows as we have entries in each observable vector, i.e. in our first implementation that should be five for $\rho_{\text{arithmetic}}, \rho_{\text{harmonic}}, \mathcal{H}_{\rho}, |\mathcal{E}|, |\mathcal{C}|$. Let T be the last iteration-number with which the algorithm terminates, i.e. where `accept` returns `.false.` for ensemble $\mathcal{E}_{T+1}$. Then, each column index in $\mathcal{O}$ represents the iteration number, i.e. column 1 contains the observable produced by iteration $T - t_{\text{observable}}$, column 2 the observable vector produced by iteration $T - t_{\text{observable}} + 1$ etc.



I suggest to use a two dimensional array. Allocate a two dimensional array $\mathcal{O}$ of $t_{\text{observable}}$ columns and as many rows as we have entries in each observable vector, i.e. in our first implementation that should be five for $\rho_{\text{arithmetic}}, \rho_{\text{harmonic}}, \mathcal{H}_{\rho}, |\mathcal{E}|, |\mathcal{C}|$. Let T be the last iteration-number with which the algorithm terminates, i.e. where `accept` returns `.false.` for ensemble $\mathcal{E}_{T+1}$. Then, each column index in $\mathcal{O}$ represents the iteration number, i.e. column 1 contains the observable produced by iteration $T - t_{\text{observable}}$, column 2 the observable vector produced by iteration $T - t_{\text{observable}} + 1$ etc.

There are a few pitfalls. E.g. if the algorithm terminates with an iteration-number smaller than $t_{\text{observable}}$ we need to make sure that the columns are correctly read out. Because in this setup column one does contain the observable vector of iteration one and not the tenth last iteration. The last observable will not be in column 10, but in the last non-zero column. We should potentially be able to return some integer that informs about how many iterations the algorithm took. Using that we can obtain the correct observable. Or, we reorder the observables and return a two dimensional array with *max* column number $t_{\text{observable}}$. If that has less columns it indicates that the algorithm terminated quickly. Talk about this with [@qwerdenkerXD](#), please.



Add a comment

Write

Preview

H B I       @   

Use Markdown to format your comment



 Paste, drop, or click to add files

 Close issue



Comment

Metadata

